DD_DIS-OP-01 - Discount Runtime Application - Developer Implementation Guide

Version: v1.0 **Bundle:** 05_billing **Companion DD:** DD_DIS-OP-01-Discount_Runtime_Application-v1.0

1. Implementation Goal

Build the runtime discount component used by billing and quote flows.

The service must let callers do two things:

- preview discounts without side effects;

- apply discounts idempotently during billing and persist the result.

The core implementation flow is:

```
sequenceDiagram
  participant BIL as BIL-01/BIL-02
  participant DIS as DIS-OP-01
  participant SIP as SIP-03
  participant PLM as PLM-CFG-04
  participant RUL as Drools/KIE
  participant WAL as BIL-05

  BIL->>DIS: POST /api/discount-runtime/apply
  DIS->>DIS: validate and hash charge context
  DIS->>DIS: check idempotency
  DIS->>SIP: GET effective assignments
  DIS->>PLM: load discount definitions
  DIS->>RUL: evaluate eligibility/stacking/caps
  DIS->>DIS: persist application, lines, decisions
  DIS->>WAL: post wallet credit when required
  DIS-->>BIL: return net amount and lines
```

2. Step 1 - Create Tables And Repositories

Create these tables first:

1. `discount_application`
2. `discount_application_line`
3. `discount_runtime_decision`
4. `discount_runtime_cap_usage`
5. `discount_runtime_replay`

Use the exact purpose from the DD:

- `discount_application` is the parent runtime attempt.

- `discount_application_line` is the money effect.

- `discount_runtime_decision` is the explanation/audit trail.

- `discount_runtime_cap_usage` protects cap limits.

- `discount_runtime_replay` controls manual/system recompute.

Minimum repository methods:

```
interface DiscountApplicationRepository {
    Optional<DiscountApplication> findByOperatorAndEventAndHashAndMode(
        String operatorCode,
        String billableEventRef,
        String chargeContextHash,
        RuntimeMode mode
    );

    Optional<DiscountApplication> findByIdempotencyKey(String operatorCode, String idempotencyKey);

    DiscountApplication save(DiscountApplication application);
```

```
}

interface CapUsageRepository {
    Optional<DiscountRuntimeCapUsage> lockByScopeAndPeriod(
        String operatorCode,
        String capScopeType,
        String capScopeRefId,
        LocalDate periodStart,
        LocalDate periodEnd
    );

    DiscountRuntimeCapUsage save(DiscountRuntimeCapUsage usage);
}
```

Important: the cap usage lookup must lock the row during `APPLY`. In PostgreSQL use `SELECT ... FOR UPDATE` or equivalent ORM pessimistic locking.

3. Step 2 - Define API Request DTOs

Both preview and apply use the same normalized request shape.

```
{
  "operatorCode": "WIK",
  "sourceModuleCode": "BIL-01",
  "mode": "APPLY",
  "billableEventCode": "RECURRING_MRC",
  "billableEventRef": "bev-20260630-sub-700045",
  "customerId": "CUS-100045",
  "accountId": "ACC-900045",
  "subscriptionId": "SUB-700045",
  "orderId": null,
  "packageRef": "pkg-fiber-100m",
  "packageVersionId": "pkv-100m-202607",
  "channelCode": "SYSTEM",
  "franchiseId": "fr-019",
  "chargeDate": "2026-06-30",
  "currencyCode": "KES",
  "baseAmount": 3000.00,
  "taxIncluded": false
}
```

Validation:

- `operatorCode`, `sourceModuleCode`, `mode`, `billableEventCode`, `billableEventRef`, `chargeDate`, `currencyCode`, and `baseAmount` are mandatory.
- At least one of `customerId`, `accountId`, `subscriptionId`, or `orderId` must be present.
- `baseAmount` must be `>= 0`.
- `mode` must match endpoint: `/preview` uses `PREVIEW`, `/apply` uses `APPLY`.
- `/apply` requires `Idempotency-Key`.

Fixed validation belongs in code, not Drools.

4. Step 3 - Implement Preview Endpoint

Endpoint:

```
POST /api/discount-runtime/preview
Content-Type: application/json
```

Flow on DIS-OP-01:

1. Validate request.
2. Normalize request and compute `chargeContextHash`.
3. Call SIP-03 effective assignment API:

```
GET /api/discount-assignments/effective?operatorCode=WIK&subscriptionId=SUB-700045&eventCode=RECURRING_MRC&onDate=2026-06-
```

4. For each returned assignment, load discount definition from PLM-CFG-04.
5. Build rule facts.

6. Run Drools eligibility and stacking rules.
7. Calculate candidate discount amounts.
8. Return lines and decisions.
9. Do not persist cap usage.
10. Do not call BIL-05.

Response sample:

```
{
  "applicationId": null,
  "status": "APPLIED",
  "baseAmount": 3000.00,
  "totalDiscountAmount": 300.00,
  "netAmount": 2700.00,
  "lines": [
    {
      "assignmentId": "da-001",
      "discountCode": "RETENTION_10PCT_3M",
      "lineType": "PRICE_REDUCTION",
      "discountAmount": 300.00,
      "currencyCode": "KES"
    }
  ],
  "decisions": [
    {
      "assignmentId": "da-001",
      "decisionResult": "ACCEPTED",
      "reasonCode": "ELIGIBLE_AND_HIGHEST_PRIORITY"
    }
  ]
}
```

5. Step 4 - Implement Apply Endpoint

Endpoint:

```
POST /api/discount-runtime/apply
Idempotency-Key: discount-apply-bev-20260630-sub-700045
Content-Type: application/json
```

On reception on DIS-OP-01:

1. Validate request and idempotency key.
2. Normalize request exactly the same way every time.
3. Compute `chargeContextHash`.
4. Check if an application already exists for `(operatorCode, billableEventRef, chargeContextHash, APPLY)`.
5. Check if the idempotency key was already used.
6. If same key and same context: return stored response.
7. If same key and different context hash: return `409 IDEMPOTENCY_KEY_CONFLICT`.
8. Start DB transaction.
9. Create `discount_application` with `DRAFT`.
10. Load candidates from SIP-03.
11. Load definitions from PLM-CFG-04.
12. Run Drools.
13. Lock and update cap usage for accepted candidates.
14. Persist accepted `discount_application_line` rows.
15. Persist accepted/rejected `discount_runtime_decision` rows.
16. Call BIL-05 for wallet-credit lines using a deterministic idempotency key.
17. Mark application `APPLIED` or `REJECTED`.

18. Commit.
19. Publish outbox events.
20. Return stored response.

Pseudo-code:

```
public DiscountRuntimeResponse apply(DiscountRuntimeRequest request, String idempotencyKey) {
    validateApplyRequest(request, idempotencyKey);

    NormalizedChargeContext context = normalizer.normalize(request);
    String contextHash = hashService.sha256(context);

    Optional<DiscountApplication> byKey =
        applicationRepo.findByIdempotencyKey(request.operatorCode(), idempotencyKey);

    if (byKey.isPresent()) {
        if (!byKey.get().chargeContextHash().equals(contextHash)) {
            throw new ConflictException("IDEMPOTENCY_KEY_CONFLICT");
        }
        return responseMapper.fromStoredApplication(byKey.get());
    }

    return tx.required(() -> {
        DiscountApplication app = applicationRepo.save(DiscountApplication.draft(context, idempotencyKey));
        RuntimeEvaluationResult result = evaluator.evaluate(context, RuntimeMode.APPLY);

        capService.reserveCaps(context, result.acceptedLines());
        lineRepo.saveAll(result.toLines(app.applicationId()));
        decisionRepo.saveAll(result.toDecisions(app.applicationId()));
        walletCreditService.postWalletCreditsIfAny(app, result.acceptedLines());

        app.finalize(result);
        outbox.publish(DiscountApplied.from(app, result));
        return responseMapper.fromStoredApplication(app);
    });
}
```

6. Step 5 - Load Effective Assignments From SIP-03

DIS-OP-01 must not query SIP-03 tables directly unless both are implemented in the same codebase and share repository contracts intentionally. Prefer the API/read-model boundary:

```
GET /api/discount-assignments/effective?operatorCode=WIK&subscriptionId=SUB-700045&eventCode=RECURRING_MRC&onDate=2026-06-
```

Map each assignment into a fact:

```
{
  "assignmentId": "da-001",
  "discountCode": "RETENTION_10PCT_3M",
  "scopeType": "SUBSCRIPTION",
  "scopeRefId": "SUB-700045",
  "priority": 100,
  "stackingGroupCode": "RECURRING_MRC",
  "validFrom": "2026-06-01",
  "validTo": "2026-08-31"
}
```

If SIP-03 is temporarily unavailable:

- `/preview` should return `503 DISCOUNT_ASSIGNMENT_LOOKUP_UNAVAILABLE`.
- `/apply` should fail safely and let BIL retry using the same idempotency key.
- Do not silently apply no discount unless Billing explicitly asked for a fail-open mode. Default is fail-closed.

7. Step 6 - Load Discount Definitions From PLM-CFG-04

For each `discountCode`, load the active definition valid on charge date.

Required fields:

```
{
  "discountCode": "RETENTION_10PCT_3M",
  "status": "ACTIVE",
  "calculationType": "PERCENT",
```

```
  "discountRate": 0.10,
  "allowedBillableEventCodes": ["RECURRING_MRC"],
  "maxAmountPerApplication": 500.00,
  "capScopeType": "ASSIGNMENT",
  "capPeriod": "MONTHLY",
  "roundingMode": "HALF_UP",
  "stackingGroupCode": "RECURRING_MRC",
  "lineType": "PRICE_REDUCTION"
}
```

If definition is missing or inactive:

- record a `discount_runtime_decision` with `REJECTED_NOT_ELIGIBLE`;
- do not produce a line;
- emit metrics for catalog mismatch.

8. Step 7 - Build Drools Facts

Create fact objects that do not expose database entities directly.

```
public record DiscountRuntimeContextFact(
    String operatorCode,
    String billableEventCode,
    String customerId,
    String accountId,
    String subscriptionId,
    String orderId,
    String packageRef,
    String packageVersionId,
    String channelCode,
    String franchiseId,
    LocalDate chargeDate,
    String currencyCode,
    BigDecimal baseAmount
) {}

public record DiscountCandidateFact(
    String assignmentId,
    String discountCode,
    String scopeType,
    String scopeRefId,
    int priority,
    String stackingGroupCode,
    String calculationType,
    BigDecimal rate,
    BigDecimal fixedAmount,
    BigDecimal maxAmountPerApplication,
    List<String> allowedBillableEventCodes
) {}

public class DiscountRuntimeDecisionFact {
    private final List<String> acceptedAssignmentIds = new ArrayList<>();
    private final Map<String, String> rejectedReasons = new HashMap<>();
    private boolean replayApprovalRequired;
    private String approvalPolicyCode;
}
```

9. Step 8 - Drools Rule Samples

9.1 Event Eligibility

```
package com.sophix.discount.runtime

rule "Reject candidate when billable event is not allowed"
when
  $ctx : DiscountRuntimeContextFact($event : billableEventCode)
  $cand : DiscountCandidateFact(!allowedBillableEventCodes.contains($event))
  $decision : DiscountRuntimeDecisionFact()
then
  $decision.reject($cand.assignmentId(), "EVENT_NOT_ALLOWED");
end
```

9.2 Subscription Scope Match

```
rule "Reject subscription discount when subscription does not match"
when
  $ctx : DiscountRuntimeContextFact($subscriptionId : subscriptionId)
  $cand : DiscountCandidateFact(scopeType == "SUBSCRIPTION", scopeRefId != $subscriptionId)
  $decision : DiscountRuntimeDecisionFact()
then
  $decision.reject($cand.assignmentId(), "SUBSCRIPTION_SCOPE_MISMATCH");
end
```

9.3 Basic Stacking Policy

```
rule "Accept highest priority candidate per stacking group"
when
  $cand : DiscountCandidateFact($group : stackingGroupCode, $id : assignmentId)
  not DiscountCandidateFact(stackingGroupCode == $group, priority > $cand.priority)
  $decision : DiscountRuntimeDecisionFact(!isRejected($id))
then
  $decision.accept($id);
end
```

9.4 Replay Approval

```
rule "Replay with financial impact requires approval"
when
  $replay : DiscountReplayFact(expectedFinancialImpact > 0)
  $decision : ReplayDecision()
then
  $decision.setApprovalRequired(true);
  $decision.setApprovalPolicyCode("DISCOUNT_RUNTIME_REPLAY");
end
```

These samples are starting points. Operators can add package, franchise, channel, campaign, or region-specific policy without changing Java code.

10. Step 9 - Calculate Amounts

After Drools returns accepted candidates, calculate amounts in code.

Rules:

- `PERCENT`: `baseAmount * rate`, then rounding.
- `FIXED_AMOUNT`: min of fixed amount and base amount unless policy allows credit.
- `FULL_WAIVER`: equal to base amount.
- `WALLET_CREDIT`: amount may exceed charge amount only if discount definition permits it.
- Never make `netAmount < 0` for `PRICE_REDUCTION` or `FEE_WAIVER`.

Sample calculation:

```
BigDecimal calculateLineAmount(BigDecimal baseAmount, DiscountDefinition def) {
    BigDecimal raw = switch (def.calculationType()) {
        case PERCENT -> baseAmount.multiply(def.discountRate());
        case FIXED_AMOUNT -> def.fixedAmount();
        case FULL_WAIVER -> baseAmount;
        case WALLET_CREDIT -> def.fixedAmount();
    };

    BigDecimal capped = applyPerApplicationCap(raw, def.maxAmountPerApplication());
    BigDecimal rounded = roundingService.round(capped, def.currencyCode(), def.roundingMode());

    if (!def.allowsCreditBeyondCharge()) {
        return rounded.min(baseAmount);
    }
    return rounded;
}
```

11. Step 10 - Cap Usage

For `APPLY`, cap usage must be updated in the same transaction as application lines.

Flow:

1. Determine cap scope and period from discount definition/rules.
2. Lock `discount_runtime_cap_usage` row.
3. If row does not exist, insert it with zero usage and lock it.
4. Check `used_amount + requested_amount <= cap_amount`.
5. If pass, increment usage.
6. If fail, reject candidate and persist decision.

Do not update cap usage for preview.

Concurrency test:

- start two apply requests for same assignment and remaining cap;
- both cannot consume the same remaining cap;
- one should apply and the other should reject or partially cap according to policy.

12. Step 11 - Wallet-Credit Lines

When an accepted line has `lineType = WALLET_CREDIT`, call BIL-05.

```
POST /api/wallets/{wallet_id}/credit
Idempotency-Key: wallet-credit-dapp-001-dal-001
Content-Type: application/json

{
  "amount": 500.00,
  "currency": "KES",
  "transactionType": "PROMOTIONAL_CREDIT",
  "reference": "dapp-001/dal-001",
  "reasonCode": "CAMPAIGN_WALLET_CREDIT"
}
```

Implementation details:

- resolve wallet through subscription/account context using BIL-05 APIs;
- use deterministic idempotency key per application line;
- store returned `walletTransactionId` on `discount_application_line`;
- retry transient failures through `discount-wallet-credit-retry`;
- if wallet credit fails after DB commit, application remains visible with retry status/metric until repaired.

13. Step 12 - Replay Flow

Request:

```
POST /api/discount-runtime/applications/{application_id}/replay
Idempotency-Key: discount-replay-dapp-001-cache-lag
Content-Type: application/json
```

DIS-OP-01 flow:

1. Load original application and charge context snapshot.
2. Validate that replay is allowed for current status.
3. Run Drools replay approval rule.
4. If approval is not required, create `discount_runtime_replay` as `APPROVED`.
5. If approval is required, call EM-CFG-04:

```
POST /api/approvals/evaluate
Content-Type: application/json

{
  "operatorCode": "WIK",
  "moduleCode": "DIS-OP-01",
  "actionCode": "DISCOUNT_RUNTIME_REPLAY",
  "subjectType": "DISCOUNT_RUNTIME_REPLAY",
```

```
  "subjectId": "drep-001",
  "context": {
    "originalApplicationId": "dapp-001",
    "expectedFinancialImpact": 300.00,
    "currencyCode": "KES",
    "reasonCode": "CACHE_LAG_REPAIR"
  }
}
```

6. If approval required, call `POST /api/approvals/requests` and store `approval_request_id`.
7. EM-CFG-04 users approve through `/api/approvals/tasks/*`.
8. EM-CFG-04 calls `/internal/discount-runtime/replays/{replay_id}/approval-outcome`.
9. DIS-OP-01 recomputes and returns adjustment instruction to billing.

Important: replay must not edit old invoice or wallet rows directly. Use billing adjustment or wallet adjustment APIs where financial correction is needed.

14. Step 13 - Outbox Events

Create an outbox row inside the same DB transaction for:

- `DiscountApplied`
- `DiscountApplicationRejected`
- `DiscountWalletCreditRequested`
- `DiscountReplayRequested`
- `DiscountReplayCompleted`

Example payload:

```
{
  "eventType": "DiscountApplied",
  "eventId": "evt-dapp-001",
  "occurredAt": "2026-06-30T00:05:01Z",
  "operatorCode": "WIK",
  "applicationId": "dapp-001",
  "subscriptionId": "SUB-700045",
  "billableEventCode": "RECURRING_MRC",
  "totalDiscountAmount": 300.00,
  "currencyCode": "KES",
  "discountCodes": ["RETENTION_10PCT_3M"]
}
```

15. Step 14 - Backoffice Support Views

Backoffice must call:

```
GET /api/discount-runtime/applications?operatorCode=WIK&subscriptionId=SUB-700045&from=2026-06-01&to=2026-06-30
```

Then:

```
GET /api/discount-runtime/applications/dapp-001
```

The detail response should include:

- charge context;
- accepted discount lines;
- rejected candidates and reasons;
- cap usage checks;
- rule package version;
- wallet transaction ids;
- invoice line refs when available.

Do not make Backoffice infer discount logic from invoice amounts alone.

16. Error Handling

Error	HTTP	Meaning
`INVALID_REQUEST`	400	Missing required field or invalid amount/date.
`IDEMPOTENCY_KEY_REQUIRED`	400	Apply request without key.
`IDEMPOTENCY_KEY_CONFLICT`	409	Same key, different context hash.
`ASSIGNMENT_LOOKUP_UNAVAILABLE`	503	SIP-03 unavailable.
`DISCOUNT_CATALOG_UNAVAILABLE`	503	PLM-CFG-04 unavailable.
`CAP_LOCK_TIMEOUT`	409/503	Cap row could not be locked in time.
`WALLET_CREDIT_FAILED`	502	BIL-05 credit failed; retry if safe.
`REPLAY_APPROVAL_REQUIRED`	202	Replay created and waiting approval.

17. Worker Jobs

Implement these as scheduled Laravel commands, Spring scheduled jobs, or queue workers depending on the chosen stack. They do not need separate pods.

Job	Command name suggestion	What it does
Outbox publisher	`discount:outbox:publish`	Publishes discount events from outbox.
Wallet retry	`discount:wallet-credit:retry`	Retries failed BIL-05 credits.
Cap reconcile	`discount:cap:reconcile`	Compares cap usage to application lines.
Replay runner	`discount:replay:run`	Runs approved replays.

If implemented in Laravel, they are Artisan commands registered in the scheduler. If implemented in Java/Spring, they are scheduled/queued services. The operational principle is the same.

18. Tests To Write First

Start with these tests:

1. Preview with no assignments returns `REJECTED` and no side effects.
2. Apply with one percent discount returns correct amount and persists line.
3. Apply retry with same idempotency key returns same response.
4. Apply retry with same key and different amount returns `409`.
5. Two discounts in same stacking group select highest priority.
6. Cap usage cannot be exceeded under concurrent requests.
7. Wallet-credit line calls BIL-05 once.
8. Replay above threshold creates EM-CFG-04 approval request.
9. EM-CFG-04 approved callback triggers replay.
10. Preview does not update cap usage or call wallet API.

19. Developer Checklist

- Tables and indexes created.
- Preview endpoint implemented.
- Apply endpoint implemented with idempotency hash check.
- SIP-03 effective assignment client implemented.

- PLM-CFG-04 discount definition client implemented.
- Drools facts and rule package integrated.
- Cap row locking implemented.
- Wallet credit integration implemented through BIL-05.
- Replay approval integration implemented through EM-CFG-04.
- Outbox events implemented.
- Backoffice read endpoints implemented.
- Unit, integration, and concurrency tests passing.